InterviewAI

AI-Powered Voice Interview Platform

Technical & Architecture Documentation

Document Overview

This document describes the architecture, modules, and data flow of the InterviewAI application — a Flask-based platform that conducts spoken, AI-driven mock interviews. It covers the voice pipeline (speech synthesis and recognition), the LLM-driven question generation and interview logic, the Flask API surface, and the front-end session and setup pages.

Field	Value
Component count	4 core modules + 2 front-end templates
Primary language	Python 3 (Flask), with HTML/CSS/JS front end
Key dependencies	Flask, LangChain, OpenAI SDK, pygame, SpeechRecognition, edge-tts, gTTS
Document scope	Codebase structure, request lifecycle, configuration, operational notes

Table of Contents

1. System Overview

InterviewAI is a self-hosted mock-interview application. A candidate configures a role and preferences on a setup page, after which the backend generates a tailored list of interview questions using a language model. The interview itself is conducted as a voice conversation: the server synthesizes speech for the AI interviewer, listens to the candidate's spoken answer via the server's microphone, transcribes it, scores it, and generates the next question or a closing statement. A feedback report is produced at the end of the session.

The system is composed of four functional layers:

- **Voice layer** (codes/voice.py) — text-to-speech via Edge TTS with a gTTS fallback, playback via pygame, and speech-to-text via SpeechRecognition + OpenAI Whisper.
- **AI/orchestration layer** (codes/ai.py) — question generation, per-answer scoring, conversational turn-taking, and end-of-session feedback reports, built on LangChain's ChatOpenAI wrapper.
- **Web/API layer** (app.py) — a Flask application exposing REST endpoints for setup, job polling, interview start/stop/pause/resume, status polling, and feedback retrieval.
- **Front end** (templates/index.html, interview.html, feedback.html) — a setup wizard and a live session view styled as a video-call-like interface, driven entirely by polling the status endpoint (no audio is streamed to the browser; all speech happens server-side).

A defining architectural trait is that **voice I/O occurs on the server**, not in the browser: the Flask process itself plays audio through pygame and listens through a physical microphone. The web page is a remote status display and control surface, not an audio client.

2. Architecture Diagram (textual)

```
Browser (setup page)
| POST /api/setup -> stores config in Flask session, spawns
| background thread: generate_questions_task()
v
Flask App (app.py) <----> codes/ai.py (LangChain / ChatOpenAI)
| |
| QUESTIONS_STORE[job_id] | writes {status, parsed questions}
| |
v
Browser (interview page)
| POST /api/interview/start -> waits on JOB_EVENTS[job_id]
| -> creates InterviewSession
| -> spawns background thread:
| _run_interview_loop(session_id)
v
_run_interview_loop (app.py)
| speak(...) -> codes/voice.py -> edge_tts / gTTS -> pygame playback
| listen_whisper_with_timeout(...) -> SpeechRecognition mic -> Whisper API
| iv_session.chat(answer) -> codes/ai.py -> score + next question
| writes to INTERVIEW_STATE[session_id] (polled every 1s by browser)
v
```

```
Browser polls GET /api/interview/status/<id> every second, renders transcript
|
v
On finish -> browser redirects to /feedback -> GET /api/get-feedback
-> codes/ai.py generate_feedback() -> LLM report (with local fallback)
```

State is held in process memory across several module-level dictionaries (QUESTIONS_STORE, SESSIONS, INTERVIEW_STATE, STARTED_JOBS, JOB_EVENTS). A background cleanup thread sweeps sessions older than SESSION_TTL_SECONDS to bound memory growth.

3. Module: codes/voice.py — Speech I/O

Handles all text-to-speech (TTS) and speech-to-text (STT) for the application. Playback uses pygame's mixer; recognition uses the SpeechRecognition library against the server's microphone, transcribed via OpenAI's Whisper API.

3.1 Text-to-Speech (speak)

- Primary engine: **edge_tts** (Microsoft Edge neural voices), run through an internal _run_async helper that wraps asyncio.run with a manual event-loop fallback.
- Fallback engine: **gTTS**, used only if Edge TTS fails or produces an empty file.
- Voice selection is randomized per session between a male and female neural voice (VOICE_MAPPING) across ten languages (English, Hindi, Telugu, Tamil, French, German, Spanish, Chinese, Japanese, Arabic).
- Generated audio is cached to disk under cache/, keyed by an MD5 hash of language, text, and voice, so identical utterances are not re-synthesized.
- speak() accepts a stop_requested_func and an optional pause_requested_func, polled at ~30 Hz during playback, so an in-progress utterance can be interrupted or paused/resumed from the Flask control endpoints.

3.2 Cache Eviction

_evict_cache_if_needed() keeps the TTS cache bounded (default 200 MB, configurable via TTS_CACHE_MAX_MB). When the cache exceeds the limit, files are removed oldest-access-time-first until the total is back under the cap. Recently reused cache entries have their mtime/atime refreshed to survive eviction longer.

3.3 Speech Recognition (listen_whisper_with_timeout)

- recognizer.pause_threshold = 1.2 — seconds of silence considered the end of an utterance; raised from an earlier 0.8s to avoid cutting off candidates during natural thinking pauses.
- adjust_for_ambient_noise() recalibrates the energy threshold against the live room on every listen attempt, rather than relying on a fixed value set at import time — this addresses both "never hears me" (quiet room) and "hears noise as speech" (loud room) failure modes.
- phrase_time_limit=45 seconds acts as a hard safety cap so silence-based end-of-speech detection is the normal stop condition, not this limit.
- Transcription is performed via ai_client.audio.transcriptions.create(model="whisper-1").
- Retries up to max_retries (default 2) on timeout, unrecognized speech, empty transcript, or any other exception; the final exception path uses log.exception to capture a full traceback, since API failures (auth, rate limits, network) previously looked identical to "the candidate wasn't heard".

3.4 Key Functions

Function	Purpose
get_voice_type()	Randomly returns "male" or "female" for a session's interviewer voice.
speak(text, lang, voice_type, stop_fn, pause_fn)	Synthesizes (or reuses cached) audio and plays it, respecting stop/pause signals.
listen_whisper_with_timeout(timeout, retries, lang, stop_fn)	Records from the microphone and returns (transcript, timed_out) after Whisper transcription.
_evict_cache_if_needed()	Bounds the on-disk TTS cache to CACHE_MAX_BYTES.
_run_async(coro)	Runs an async coroutine synchronously from a sync call site.

4. Module: codes/ai.py — LLM Orchestration

Contains all LangChain / OpenAI interaction: question generation, the live interview conversation loop, per-answer scoring, and the final feedback report. Uses ChatOpenAI from langchain_openai, model configurable via INTERVIEWAI_MODEL (default gpt-4o-mini).

4.1 Shared Infrastructure

- _LLM_CACHE — one ChatOpenAI client per temperature value, reused across calls instead of re-instantiating (and re-opening an HTTP connection pool) on every request; guarded by a lock for thread safety.
- _invoke_with_retry() — wraps llm.invoke() with exponential backoff (0.6s × 2^attempt) across LLM_MAX_RETRIES (default 2) attempts, so transient network errors or 429/5xx responses do not abort an interview.
- _FENCE_RE — a compiled regular expression that strips Markdown code fences from LLM output before JSON parsing; compiled once at import time rather than per call.

4.2 Question Generation (generate_questions_task)

- Runs in a background thread, writing progress into the module-level QUESTIONS_STORE[job_id] dictionary (status: processing / completed / failed).
- Builds a prompt from role details (title, type, industry, experience, JD excerpt, skills, focus, difficulty, tone, language) and asks the model for a JSON array of question objects, each with question, hint, and type.
- If the requested AI question count is 0 (i.e. the candidate supplied only custom questions), the LLM call is skipped entirely.
- Parsing is attempted up to twice: if the first response is not valid JSON, a second attempt is made at a lower temperature (0.4) with an explicit corrective instruction.
- Custom candidate-supplied questions are appended to the parsed list with type: "custom".

4.3 InterviewSession — Live Conversation State

Each active interview is represented by an InterviewSession instance held in the module-level SESSIONS dictionary, keyed by session ID.

Method	Behavior
start()	Produces an opening greeting plus the first question, seeded into an in-memory chat history (InMemoryChatMessageHistory).
chat(user_answer)	Scores the current answer, records it in session.history, advances the question index, and either produces a transition to the next question or a closing statement if the interview is complete. Guarded by a threading.Lock.

Method	Behavior
_invoke(user_prompt)	Sends the system prompt plus running chat history plus a new turn-specific instruction to the interviewer LLM.
_score_answer(question, answer, hint)	Calls a separate temperature-0 scorer LLM that must return {"score": 1-10, "feedback": "..."}; on any failure, falls back to a neutral score of 5.

The interviewer system prompt embeds the full internal question list up front (marked as not to be read aloud) along with behavioral rules: ask one question at a time, give brief feedback before transitioning, stay in character, and never mention being an AI.

4.4 Feedback Report (generate_feedback)

- Builds a full question/answer/score/feedback log and asks the LLM for a structured JSON report: overall_score, recommendation (Hire / Maybe / No Hire), summary, strengths, improvements, and a per-question score breakdown.
- If the LLM call or JSON parse fails, _fallback_feedback() computes a report locally from the already-recorded per-question scores (simple average, recommendation thresholds at ≥7 / ≥4.5), so the candidate is never left with a hard failure. The fallback report is flagged with is_fallback: true.
- Feedback is cached on the session object (_feedback_cache) so repeated requests do not re-invoke the LLM.

5. Module: app.py — Flask Application & API

The Flask entry point. Serves the three HTML templates and exposes the JSON API that drives setup, question generation polling, the live interview loop, and feedback retrieval. Runs the built-in dev server with threaded=True (a production deployment should instead sit behind gunicorn/uWSGI, per an in-code comment).

5.1 In-Memory State

Structure	Purpose
INTERVIEW_STATE	session_id -> live status dict (status, transcript, question_number, pause/stop flags, timestamps), polled by the front end every second.
STARTED_JOBS	job_id -> session_id, preventing a page refresh from spawning a second voice thread for the same job.
JOB_EVENTS	job_id -> threading.Event, set once question generation completes (success or failure), enabling an event-based wait rather than sleep-polling.
QUESTIONS_STORE / SESSIONS	Imported from codes.ai; hold generated questions and live InterviewSession objects respectively.

5.2 Background Session Cleanup

A daemon thread (_cleanup_loop) wakes every CLEANUP_INTERVAL_SECONDS (10 minutes) and removes sessions whose status is terminal (finished / stopped / error) and whose finished_at is older than SESSION_TTL_SECONDS (default 3600s), purging their entries from INTERVIEW_STATE, SESSIONS, STARTED_JOBS, QUESTIONS_STORE, and JOB_EVENTS to bound memory growth over a long-running server process.

5.3 API Endpoints

Method	Path	Description
GET	/	Renders the setup page (index.html).
GET	/interview	Renders the live session page; redirects to "/" if no config/job is in the Flask session.
GET	/feedback	Renders the feedback page; redirects to "/" if no session ID is stored.
GET	/health	Returns a JSON health payload with active and total tracked session counts.

Method	Path	Description
POST	/api/setup	Validates and stores interview configuration in the Flask session; spawns a background thread to generate questions; returns a job_id.
GET	/api/status/	Returns the current question-generation job status (processing / completed / failed) and parsed questions.
POST	/api/interview/start	Waits (event-based, up to 45s) for question generation to finish, creates an InterviewSession, and spawns the background voice loop thread.
GET	/api/interview/status/	Returns the live status dict for the session (used by the 1-second poll loop).
POST	/api/interview/chat	Text-only fallback endpoint; explicitly not used by the voice-driven UI and must not be called concurrently with an active voice loop for the same session.
GET/POST	/api/get-feedback	Generates (or returns cached) the end-of-session feedback report.
POST	/api/interview/stop	Sets stop_requested on the session's state, ending the voice loop.
POST	/api/interview/pause	Sets pause_requested; the voice loop blocks in _pause_gate() until cleared.
POST	/api/interview/resume	Clears pause_requested / paused.

5.4 Validation & Limits

- _validate_setup_payload() requires a non-empty job title and a question count between MIN_QUESTIONS (1) and MAX_QUESTIONS (30).
- JOB_WAIT_TIMEOUT_SECONDS = 45 bounds how long /api/interview/start will wait on question generation before returning HTTP 504.
- SESSION_TTL_SECONDS (default 3600, overridable via environment variable) controls how long a finished session remains queryable before cleanup.

5.5 The Interview Loop (_run_interview_loop)

Executed in its own daemon thread per session. Repeatedly: gates on any pause request, speaks the current AI message, transitions to a "listening" status, calls listen_whisper_with_timeout, and — if a transcript was captured — passes it into iv_session.chat(), updates state, and speaks the result. On empty or timed-out input, it asks the candidate to repeat rather than failing the session. Any unhandled exception sets the session status to error with the exception message.

6. Front End: Setup Page & Interview Session Page

6.1 Setup Page (index.html)

- A three-section configuration form: role information (title, type, industry, experience, JD text), skills & focus (tag input plus a focus pill-selector: technical / behavioral / system-design / situational / mixed), and session preferences (language, tone, difficulty, question count slider 3–20, optional custom questions textarea, and a "show answer hints" toggle).
- Includes a restored-draft banner and an inline error banner, both hidden by default and toggled by the accompanying script.js.
- Submits to POST /api/setup, then presumably polls /api/status/ before navigating to /interview (driven by the external script.js, not shown in the reviewed files).

6.2 Interview Session Page (interview.html)

Styled as a video-call-like single tile showing an animated "AI Interviewer" avatar, with a right-hand transcript panel, a header with a live timer and question counter, and pause/end controls. All actual audio I/O happens server-side; this page is a pure status mirror and control surface.

- On load, calls POST /api/interview/start with the job ID embedded from the Flask template context, then begins polling GET /api/interview/status/ once per second.
- Renders status-driven UI states: starting, speaking, listening, thinking, finished, stopped, error — reflected in avatar animation, status text, and a typing indicator.
- New transcript entries (with per-answer score chips and feedback) are appended incrementally as they appear in the polled state, tracked via a renderedCount cursor so entries are not duplicated.
- Pause/Resume calls /api/interview/pause or /api/interview/resume and shows a full-screen pause overlay while state.paused is true.
- A connection-loss banner appears after two consecutive failed polls and offers a manual retry button; the poll loop itself keeps running independently.
- Ending the interview opens a confirmation modal, then calls /api/interview/stop and redirects to /feedback. On natural completion (status === "finished") the page auto-redirects after 1.5s.
- A beforeunload handler warns on accidental tab close while a session ID is active.

7. Request Lifecycle (End-to-End)

- **Setup** — Candidate fills out the setup form and submits. POST /api/setup validates the payload, stores config in the Flask session, and starts a background thread that calls generate_questions_task.
- **Question generation** — The AI module builds a role-specific prompt and requests a JSON array of questions from the LLM, retrying once on malformed JSON; the result (or failure) is written to QUESTIONS_STORE and the associated event is set.
- **Interview start** — The interview page calls POST /api/interview/start, which waits on the generation event (up to 45s), builds an InterviewSession, initializes live state, and starts the voice-loop thread.
- **Turn cycle** — For each question: the loop speaks the AI's message (codes/voice.speak), listens for the candidate's spoken answer (listen_whisper_with_timeout), scores the answer and produces the next message (InterviewSession.chat), and updates INTERVIEW_STATE, which the browser reflects within one polling interval.
- **Completion** — After the final answer, the session is marked finished; the browser detects this on its next poll and redirects to /feedback.
- **Feedback** — GET/POST /api/get-feedback triggers generate_feedback, which produces a structured hire/no-hire report from the full Q&A; log, falling back to a locally computed summary if the LLM call fails.

8. Configuration & Environment Variables

Variable	Default	Purpose
OPENAI_API_KEY	required	OpenAI API key; both codes/ai.py and codes/voice.py raise at import time if unset.
INTERVIEWAI_MODEL	gpt-4o-mini	Chat model used for question generation, interview turns, scoring, and feedback.
TTS_CACHE_MAX_MB	200	Maximum size in MB of the on-disk TTS audio cache before oldest entries are evicted.
SESSION_TTL_SECONDS	3600	How long a finished/stopped/errored session remains before background cleanup removes it.
SECRET_KEY	dev-secret-change-in-prod	Flask session signing key; should be overridden in any real deployment.
LOG_LEVEL	INFO	Root logging level for the application logger.
FLASK_DEBUG	1	Controls whether the Flask dev server runs in debug mode.

Hard-coded operational limits worth tracking separately from environment configuration: MAX_QUESTIONS = 30, MIN_QUESTIONS = 1, JOB_WAIT_TIMEOUT_SECONDS = 45, CLEANUP_INTERVAL_SECONDS = 600, LLM_MAX_RETRIES = 2, and voice-recognition tuning (pause_threshold = 1.2s, phrase_time_limit = 45s).

9. Operational Notes, Limitations & Risks

9.1 Deployment

- The Flask built-in dev server (app.run(..., threaded=True)) is used directly; the in-code comment already flags that a production deployment should run behind gunicorn or uWSGI instead.
- Because voice playback and microphone capture happen on the **server** process, this architecture is only appropriate for a single-machine, single-user deployment (e.g. a kiosk or local desktop app) — it does not generalize to multiple concurrent remote users sharing one server, since they would all be fighting over one physical mic and speaker.

9.2 State & Persistence

- All session, question, and interview state is held in process memory. A server restart loses every in-progress and completed-but-uncleaned session; there is no database-backed persistence layer in the reviewed code.
- The background cleanup thread only removes sessions that have reached a terminal status; a session stuck in a non-terminal status (e.g. due to a crashed voice thread) would not be cleaned up automatically.

9.3 Error Handling

- LLM calls are wrapped in retry-with-backoff logic; question generation and feedback generation both have explicit fallback behavior (skip-if-zero for questions with no AI-generated portion, and a locally computed report for feedback) so a single LLM hiccup does not hard-fail a session.
- Speech recognition failures (timeout, unrecognized speech, empty transcript) are retried up to max_retries before the loop asks the candidate to repeat their answer, rather than aborting the interview.

9.4 Security Notes

- SECRET_KEY defaults to a literal development value ("dev-secret-change-in-prod"); this must be overridden via environment variable in any non-local deployment, since it signs the Flask session cookie.
- CORS is enabled broadly (CORS(app, supports_credentials=True)) with no origin allow-list visible in the reviewed code.
- /api/interview/chat is explicitly documented in-code as a fallback not used by the current UI, and calling it concurrently with the background voice loop for the same session would race on the shared InterviewSession object.

9.5 Front-End / Back-End Coupling

- The front end has no direct knowledge of audio; it is entirely a polling client over JSON status. This makes the UI resilient to connection drops (a banner and retry button appear after repeated poll failures) but means the actual interview experience is undiscoverable from the browser alone — a tester must be at the server machine to hear/speak.

- The setup and interview pages reference an external /static/script.js and /static/style1.css that were not included in the reviewed file set, so client-side draft-saving, validation, and template logic referenced by the setup page (draft banner, save-as-template) could not be fully verified here.